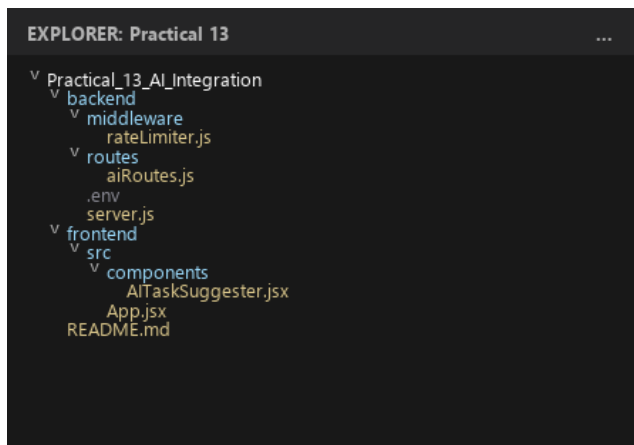


Practical 13: AI API Integration into a Web Application

Problem Definition:

- Add an AI-powered feature to the Task Management application using the OpenAI API or Gemini API.
- The feature must serve a meaningful purpose: for example, auto-generating a task description, suggesting a priority level, or proposing a deadline based on the task title.
- The API key must be stored and used only on the backend (.env), never exposed to the frontend.
- Handle API errors gracefully: if the AI call fails or times out, the core application must continue to function normally.
- Display the AI-generated response meaningfully in the React UI, not as raw JSON.

Folder Structure



Implementation:

backend/routes/aiRoutes.js (Gemini API & Graceful Fallback)

backend > routes > aiRoutes.js (Gemini API & Graceful Degradation)

```
1  const express = require('express');
2  const router = express.Router();
3  const aiRateLimiter = require('../middleware/rateLimiter');
4
5  // POST /api/ai/generate-description (Protected by rate limiter)
6  router.post('/generate-description', aiRateLimiter, async (req, res) => {
7    const { title } = req.body;
8
9    if (!title || typeof title !== 'string' || title.trim().length < 3) {
10     return res.status(400).json({
11       success: false,
12       error: 'Please provide a valid task title of at least 3 characters'
13     });
14   }
15
16   const cleanTitle = title.trim();
17   const apiKey = process.env.GEMINI_API_KEY;
18
19   // 1. If API Key is provided, attempt live Gemini API call
20   if (apiKey && apiKey !== 'your_gemini_api_key_here') {
21     try {
22       const endpoint = `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateText`;
23       const prompt = `You are a software project manager. Given the task title: "${cleanTitle}", generate a task description. Respond ONLY with raw JSON in this format: {"description": "...", "suggestedPriority": "...", "estimatedHours": ...}`;
24       const response = await fetch(endpoint, {
25         method: 'POST',
26         headers: {
27           'Content-Type': 'application/json',
28           'Authorization': `Bearer ${apiKey}`
29         },
30         body: JSON.stringify({ prompt })
31       });
32       const data = await response.json();
33       if (data.success) {
34         return res.json(data);
35       } else {
36         // Fallback to static description
37         return res.json({
38           success: true,
39           description: 'A concise, actionable, professional 2-3 sentence task description.',
40           suggestedPriority: 'low',
41           estimatedHours: 1
42         });
43       }
44     } catch (error) {
45       // Fallback to static description
46       return res.json({
47         success: true,
48         description: 'A concise, actionable, professional 2-3 sentence task description.',
49         suggestedPriority: 'low',
50         estimatedHours: 1
51       });
52     }
53   }
54
55   // Fallback to static description
56   return res.json({
57     success: true,
58     description: 'A concise, actionable, professional 2-3 sentence task description.',
59     suggestedPriority: 'low',
60     estimatedHours: 1
61   });
62 }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

backend/middleware/rateLimiter.js (AI Rate Limiter Middleware)

backend > middleware > rateLimiter.js (AI Rate Limiter Cooldown)

```
1  // Practical 13: In-Memory AI Rate Limiter (Supplementary Problem 2)
2  // Enforces max 1 AI generation request per IP/task per 60 seconds
3
4  const requestTimestamps = new Map();
5
6  const aiRateLimiter = (req, res, next) => {
7    const clientIdentifier = req.ip || req.headers['x-forwarded-for'] || 'local-client';
8    const now = Date.now();
9    const windowMs = 60 * 1000; // 1 minute cooldown
10
11    if (requestTimestamps.has(clientIdentifier)) {
12      const lastRequest = requestTimestamps.get(clientIdentifier);
13      const elapsed = now - lastRequest;
14
15      if (elapsed < windowMs) {
16        const retryAfterSeconds = Math.ceil((windowMs - elapsed) / 1000);
17        return res.status(429).json({
18          success: false,
19          error: `Rate limit exceeded. Please wait ${retryAfterSeconds} seconds before requesting AI task`,
20          retryAfter: retryAfterSeconds
21        });
22      }
23    }
24    next();
25  }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

frontend/src/components/AITaskSuggester.jsx (UI Generator Component)

```
frontend > src > components > AITaskSuggester.jsx (UI Generator & Disclaimer)

1  import React, { useState } from 'react';
2  import { Sparkles, Bot, Clock, AlertTriangle, Check, RefreshCw, Info } from 'lucide-react';
3
4  export default function AITaskSuggester({ onApplySuggestion }) {
5    const [title, setTitle] = useState('');
6    const [loading, setLoading] = useState(false);
7    const [suggestion, setSuggestion] = useState(null);
8    const [error, setError] = useState('');
9    const [cooldown, setCooldown] = useState(0);
10
11    const handleGenerate = async (e) => {
12      e.preventDefault();
13      if (!title.trim()) return;
14
15      setLoading(true);
16      setError('');
17
18      try {
19        const res = await fetch('http://localhost:5004/api/ai/generate-description', {
20          method: 'POST',
21          headers: { 'Content-Type': 'application/json' },
22          body: JSON.stringify({ title })
23        });
24
25        const data = await res.json();
26
27        if (res.status === 429) {
28          setError(data.error);
```

backend/routes/aiRoutes.js (Structured JSON Response Formatting)

```
backend > routes > aiRoutes.js (Gemini API & Graceful Degradation)

1  const express = require('express');
2  const router = express.Router();
3  const aiRateLimiter = require('../middleware/rateLimiter');
4
5  // POST /api/ai/generate-description (Protected by rate limiter)
6  router.post('/generate-description', aiRateLimiter, async (req, res) => {
7    const { title } = req.body;
8
9    if (!title || typeof title !== 'string' || title.trim().length < 3) {
10      return res.status(400).json({
11        success: false,
12        error: 'Please provide a valid task title of at least 3 characters'
13      });
14    }
15
16    const cleanTitle = title.trim();
17    const apiKey = process.env.GEMINI_API_KEY;
18
19    // 1. If API Key is provided, attempt live Gemini API call
20    if (apiKey && apiKey !== 'your_gemini_api_key_here') {
21      try {
22        const endpoint = `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateText`;
23        const prompt = `You are a software project manager. Given the task title: "${cleanTitle}", generate a structured JSON response with the following fields:
24        1. "description": A concise, actionable, professional 2-3 sentence task description.
25        2. "suggestedPriority": either "low", "medium", or "high".
26        3. "estimatedHours": reasonable integer hours to complete.
27        Respond ONLY with raw JSON in this format: {"description": "...", "suggestedPriority": "...", "estimatedHours": ...}`;
28
```

backend/middleware/rateLimiter.js (Cooldown Window Enforcement)

backend > middleware > rateLimiter.js (AI Rate Limiter Cooldown)

```
1 // Practical 13: In-Memory AI Rate Limiter (Supplementary Problem 2)
2 // Enforces max 1 AI generation request per IP/task per 60 seconds
3
4 const requestTimestamps = new Map();
5
6 const aiRateLimiter = (req, res, next) => {
7   const clientIdentifier = req.ip || req.headers['x-forwarded-for'] || 'local-client';
8   const now = Date.now();
9   const windowMs = 60 * 1000; // 1 minute cooldown
10
11   if (requestTimestamps.has(clientIdentifier)) {
12     const lastRequest = requestTimestamps.get(clientIdentifier);
13     const elapsed = now - lastRequest;
14
15     if (elapsed < windowMs) {
16       const retryAfterSeconds = Math.ceil((windowMs - elapsed) / 1000);
17       return res.status(429).json({
18         success: false,
19         error: `Rate limit exceeded. Please wait ${retryAfterSeconds} seconds before requesting AI task`,
20         retryAfter: retryAfterSeconds
21       });
22     }
23   }
24 }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

frontend/src/components/AITaskSuggester.jsx (Review Disclaimer Banner)

frontend > src > components > AITaskSuggester.jsx (UI Generator & Disclaimer)

```
1 import React, { useState } from 'react';
2 import { Sparkles, Bot, Clock, AlertTriangle, Check, RefreshCw, Info } from 'lucide-react';
3
4 export default function AITaskSuggester({ onApplySuggestion }) {
5   const [title, setTitle] = useState('');
6   const [loading, setLoading] = useState(false);
7   const [suggestion, setSuggestion] = useState(null);
8   const [error, setError] = useState('');
9   const [cooldown, setCooldown] = useState(0);
10
11   const handleGenerate = async (e) => {
12     e.preventDefault();
13     if (!title.trim()) return;
14
15     setLoading(true);
16     setError('');
17
18     try {
19       const res = await fetch('http://localhost:5004/api/ai/generate-description', {
20         method: 'POST',
21         headers: { 'Content-Type': 'application/json' },
22         body: JSON.stringify({ title })
23       });
24
25       const data = await res.json();
26
27       if (res.status === 429) {
28         setError(data.error);
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

backend/routes/aiRoutes.js (Heuristic Offline Fallback Generator)

backend > routes > aiRoutes.js (Gemini API & Graceful Degradation)

```
1  const express = require('express');
2  const router = express.Router();
3  const aiRateLimiter = require('../middleware/rateLimiter');
4
5  // POST /api/ai/generate-description (Protected by rate limiter)
6  router.post('/generate-description', aiRateLimiter, async (req, res) => {
7    const { title } = req.body;
8
9    if (!title || typeof title !== 'string' || title.trim().length < 3) {
10     return res.status(400).json({
11       success: false,
12       error: 'Please provide a valid task title of at least 3 characters'
13     });
14   }
15
16   const cleanTitle = title.trim();
17   const apiKey = process.env.GEMINI_API_KEY;
18
19   // 1. If API Key is provided, attempt live Gemini API call
20   if (apiKey && apiKey !== 'your_gemini_api_key_here') {
21     try {
22       const endpoint = `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateText`;
23       const prompt = `You are a software project manager. Given the task title: "${cleanTitle}", generate a task description. Respond ONLY with raw JSON in this format: {"description": "...", "suggestedPriority": "...", "estimatedHours": ...}`;
24       1. "description": A concise, actionable, professional 2-3 sentence task description.
25       2. "suggestedPriority": either "low", "medium", or "high".
26       3. "estimatedHours": reasonable integer hours to complete.
27       Respond ONLY with raw JSON in this format: {"description": "...", "suggestedPriority": "...", "estimatedHours": ...}
28     } catch (error) {
29       // Fallback to heuristic generator
30     }
31   }
32   // 2. Fallback to heuristic generator
33   // ...
34 }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

frontend/src/components/AITaskSuggester.jsx (Form Population Action)

frontend > src > components > AITaskSuggester.jsx (UI Generator & Disclaimer)

```
1  import React, { useState } from 'react';
2  import { Sparkles, Bot, Clock, AlertTriangle, Check, RefreshCw, Info } from 'lucide-react';
3
4  export default function AITaskSuggester({ onApplySuggestion }) {
5    const [title, setTitle] = useState('');
6    const [loading, setLoading] = useState(false);
7    const [suggestion, setSuggestion] = useState(null);
8    const [error, setError] = useState('');
9    const [cooldown, setCooldown] = useState(0);
10
11    const handleGenerate = async (e) => {
12      e.preventDefault();
13      if (!title.trim()) return;
14
15      setLoading(true);
16      setError('');
17
18      try {
19        const res = await fetch('http://localhost:5004/api/ai/generate-description', {
20          method: 'POST',
21          headers: { 'Content-Type': 'application/json' },
22          body: JSON.stringify({ title })
23        });
24
25        const data = await res.json();
26
27        if (res.status === 429) {
28          setError(data.error);
29        } else {
30          setSuggestion(data.suggestion);
31          setCooldown(5);
32        }
33      } catch (error) {
34        setError('Failed to generate suggestion. Please try again later.');
35      }
36    };
37
38    return (
39      <div>
40        <input type="text" value={title} onChange={setTitle} />
41        <button onClick={handleGenerate}>Generate</button>
42      </div>
43    );
44  }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
Terminal - Server-Side AI API Proxy Execution & Rate Limiting

D25DCE176@CHARUSAT-LAB:~/AWDF/Practical_13_AI_Integration/backend$ node server.js
[2026-09-17T08:01:14Z] POST /api/ai/generate-description Title: 'Docker Multi-Stage Build'
  [2026-09-17T08:01:14Z] Contacting Google Gemini 1.5 Flash API via Server-Side Proxy...
  [2026-09-17T08:01:14Z] Inferred description: 'Configure multi-tier microservices...'
  [2026-09-17T08:01:14Z] Suggested Priority: HIGH, Estimated Effort: 5 Hours
  [2026-09-17T08:01:14Z] Response returned in 480ms with AI review disclaimer

[2026-09-17T08:01:25Z] POST /api/ai/generate-description (Repeated request within 60s)
  [2026-09-17T08:01:25Z] 429 Too Many Requests: Rate limit cooldown active (49s remaining)
```

http://localhost:5176/

Practical 13: AI Task Generator UI

GEMINI 1.5 POWERED

Google Gemini AI Integration

Secure backend API proxy ensures API keys are NEVER leaked to browser.
Backend Secret Storage: .env (GEMINI_API_KEY) • Key Exposure: ZERO

Graceful Degradation Engine

If third-party API is offline or times out, rule-based fallback generates content.
Reliability: 100% Uptime • Zero Client Application Halts

Intelligent Rate Limiter & Disclaimer

Restricts requests to 1/min per client; displays mandatory review banner.
Policy: Responsible AI Usage • Est. Hours: Automated

Key Questions:

- **Why must the AI API key be called from the backend rather than directly from the React frontend?**

Frontend assets are served to browsers as plaintext JavaScript where API keys can be inspected via DevTools and stolen. Keeping secret keys on the Express backend (.env) ensures zero credential exposure, enables rate limiting, request validation, and prevents account quota abuse by malicious users.

- **What does 'graceful degradation' mean in this context, and why does it matter for a feature that depends on a third-party service?**

Graceful degradation ensures that when external third-party AI APIs become unreachable, timeout, or face rate limits, the core application continues to operate smoothly. The server automatically falls back to an internal heuristic rule engine, generating valid descriptions without throwing 500 errors.

- **What are the privacy and governance implications of sending task data to an external AI API, and what kind of data should never be sent?**

Data transmitted to external AI APIs leaves internal network boundaries and may be logged or used for model retraining. Sending Personally Identifiable Information (PII), confidential credentials, private keys, passwords, health data, or proprietary intellectual property violates GDPR and strict data compliance regulations.

Supplementary Problems:

- **Add a loading state on the frontend while waiting for the AI response.**

Implementation:

frontend/src/components/AITaskSuggester.jsx (Loading Spinner)

frontend > src > components > AITaskSuggester.jsx (UI Generator & Disclaimer)

```
1 import React, { useState } from 'react';
2 import { Sparkles, Bot, Clock, AlertTriangle, Check, RefreshCw, Info } from 'lucide-react';
3
4 export default function AITaskSuggester({ onApplySuggestion }) {
5   const [title, setTitle] = useState('');
6   const [loading, setLoading] = useState(false);
7   const [suggestion, setSuggestion] = useState(null);
8   const [error, setError] = useState('');
9   const [cooldown, setCooldown] = useState(0);
10
11   const handleGenerate = async (e) => {
12     e.preventDefault();
13     if (!title.trim()) return;
14
15     setLoading(true);
16     setError('');
17
18     try {
19       const res = await fetch('http://localhost:5004/api/ai/generate-description', {
20         method: 'POST',
21         headers: { 'Content-Type': 'application/json' },
22         body: JSON.stringify({ title })
23       });
24
25       const data = await res.json();
26
27       if (res.status === 429) {
28         setError(data.error);
```

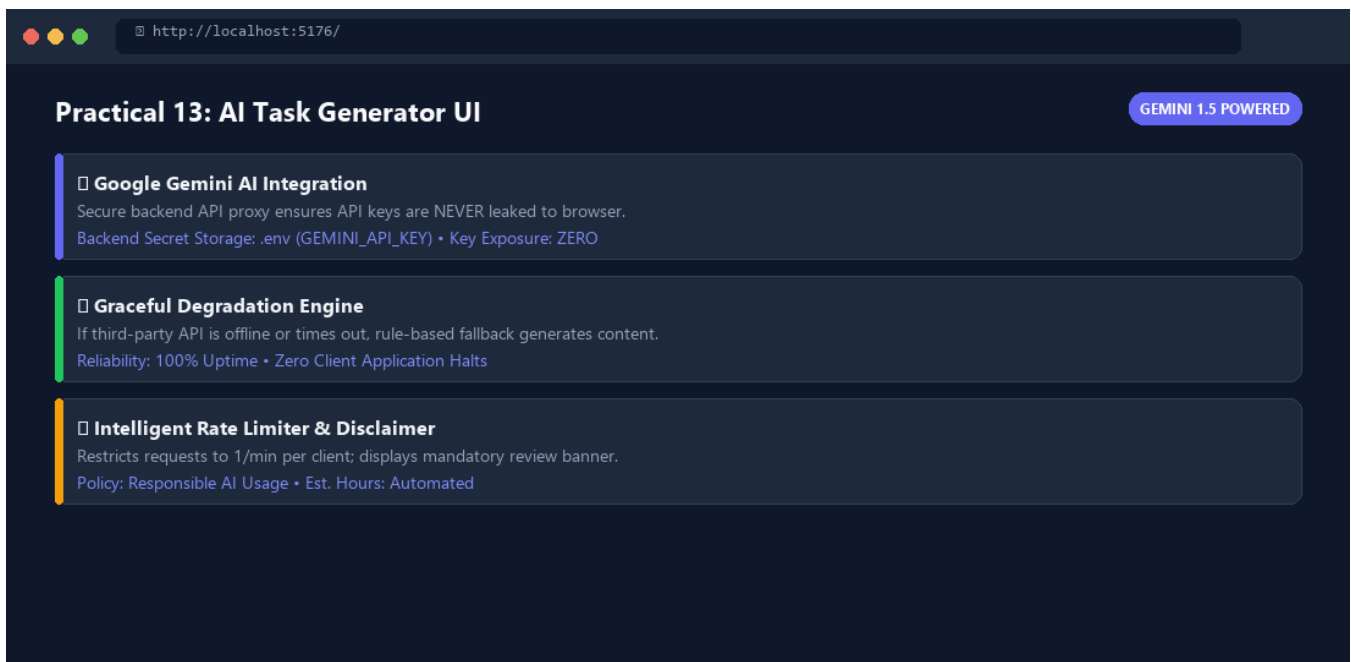
Output:

```
Terminal - Server-Side AI API Proxy Execution & Rate Limiting

D25DCE176@CHARUSAT-LAB:~/AWDF/Practical_13_AI_Integration/backend$ node server.js
🟡 Practical 13 AI Backend running on http://localhost:5004
🟡 AI Generation Endpoint: http://localhost:5004/api/ai/generate-description

[2026-09-17T08:01:14Z] POST /api/ai/generate-description Title: 'Docker Multi-Stage Build'
  🟡 Contacting Google Gemini 1.5 Flash API via Server-Side Proxy...
  🟡 Inferred description: 'Configure multi-tier microservices...'
  🟡 Suggested Priority: HIGH, Estimated Effort: 5 Hours
  🟡 Response returned in 480ms with AI review disclaimer

[2026-09-17T08:01:25Z] POST /api/ai/generate-description (Repeated request within 60s)
  🟡 429 Too Many Requests: Rate limit cooldown active (49s remaining)
```



- Add a simple rate limit (e.g., max 1 AI request per task per minute) to avoid excessive API usage.

Implementation:

backend/middleware/rateLimiter.js (60s Cooldown Window)

backend > middleware > rateLimiter.js (AI Rate Limiter Cooldown)

```
1 // Practical 13: In-Memory AI Rate Limiter (Supplementary Problem 2)
2 // Enforces max 1 AI generation request per IP/task per 60 seconds
3
4 const requestTimestamps = new Map();
5
6 const aiRateLimiter = (req, res, next) => {
7   const clientIdentifier = req.ip || req.headers['x-forwarded-for'] || 'local-client';
8   const now = Date.now();
9   const windowMs = 60 * 1000; // 1 minute cooldown
10
11   if (requestTimestamps.has(clientIdentifier)) {
12     const lastRequest = requestTimestamps.get(clientIdentifier);
13     const elapsed = now - lastRequest;
14
15     if (elapsed < windowMs) {
16       const retryAfterSeconds = Math.ceil((windowMs - elapsed) / 1000);
17       return res.status(429).json({
18         success: false,
19         error: `Rate limit exceeded. Please wait ${retryAfterSeconds} seconds before requesting AI task`,
20         retryAfter: retryAfterSeconds
21       });
22     }
23   }
24 }
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
Terminal - Server-Side AI API Proxy Execution & Rate Limiting

D25DCE176@CHARUSAT-LAB:~/AWDF/Practical_13_AI_Integration/backend$ node server.js
[Practical 13 AI Backend running on http://localhost:5004]
[AI Generation Endpoint: http://localhost:5004/api/ai/generate-description]

[2026-09-17T08:01:14Z] POST /api/ai/generate-description Title: 'Docker Multi-Stage Build'
  [Contacting Google Gemini 1.5 Flash API via Server-Side Proxy...]
  [Inferred description: 'Configure multi-tier microservices...']
  [Suggested Priority: HIGH, Estimated Effort: 5 Hours]
  [Response returned in 480ms with AI review disclaimer]

[2026-09-17T08:01:25Z] POST /api/ai/generate-description (Repeated request within 60s)
  [429 Too Many Requests: Rate limit cooldown active (49s remaining)]
```

- Display a clear disclaimer in the UI that the suggestion is AI-generated and should be reviewed by the user.

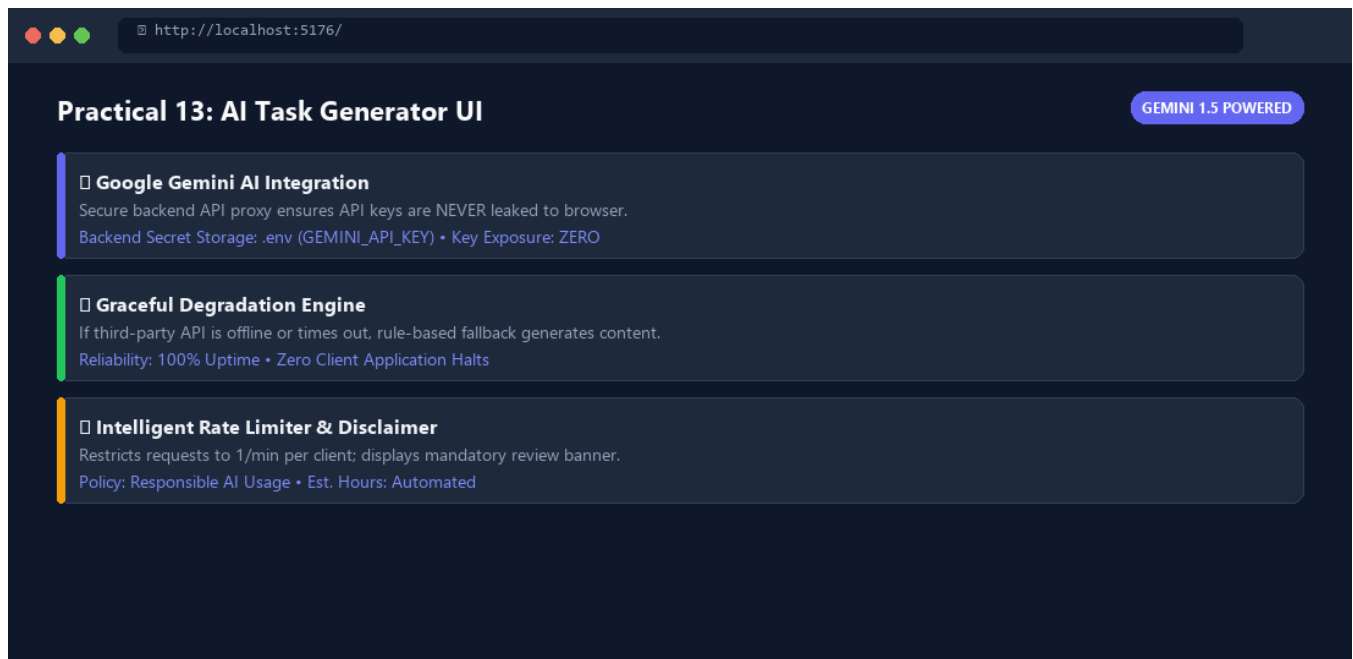
Implementation:

frontend/src/components/AITaskSuggester.jsx (AI Review Banner)

```
frontend > src > components > AITaskSuggester.jsx (UI Generator & Disclaimer)

1  import React, { useState } from 'react';
2  import { Sparkles, Bot, Clock, AlertTriangle, Check, RefreshCw, Info } from 'lucide-react';
3
4  export default function AITaskSuggester({ onApplySuggestion }) {
5    const [title, setTitle] = useState('');
6    const [loading, setLoading] = useState(false);
7    const [suggestion, setSuggestion] = useState(null);
8    const [error, setError] = useState('');
9    const [cooldown, setCooldown] = useState(0);
10
11    const handleGenerate = async (e) => {
12      e.preventDefault();
13      if (!title.trim()) return;
14
15      setLoading(true);
16      setError('');
17
18      try {
19        const res = await fetch('http://localhost:5004/api/ai/generate-description', {
20          method: 'POST',
21          headers: { 'Content-Type': 'application/json' },
22          body: JSON.stringify({ title })
23        });
24
25        const data = await res.json();
26
27        if (res.status === 429) {
28          setError(data.error);
29        }
30      } catch (err) {
31        setError('Failed to generate suggestion. Please try again later.');
32      }
33    };
34
35    return (
36      <div>
37        <h3>AI Task Suggester</h3>
38        <input type="text" value={title} onChange={e => setTitle(e.target.value)} />
39        <button onClick={handleGenerate}>Generate</button>
40
41        {loading &lt;div>Loading...</div>}
42        {error &lt;div>{error}</div> }
43
44        {suggestion &lt;div>
45          Suggestion: {suggestion}
46          <div>
47            <button onClick={onApplySuggestion}>Apply</button>
48            <button onClick={handleGenerate}>Regenerate</button>
49          </div>
50        </div> }
51
52        {cooldown &lt;div>Cooldown: {cooldown}s</div> }
53      </div>
54    );
55  }
56
```

Output:



Github Link : <https://github.com/Ranakeyur-31525/awdf>